

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

**Duration: 1 Hour
Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

S PRAVEEN (192524239)

ARTIFICIAL INTELLIGENCE — ASSESSMENT TOOL 2

Name: S PRAVEEN Reg. No: 192524239

1. Scenario: AI Agent for a Simulated Auction-Bidding Game

i. How Minimax models the problem

The auction can be represented as a two-player, adversarial decision problem. The fintech startup's AI is the maximizing player, while the competing automated bidder is the minimizing player. A game state can contain the current market conditions, available budget, remaining auction rounds, the current highest bid, item value estimates, and information revealed by previous bids. From each state, the AI generates legal actions such as bidding, increasing a bid, waiting, or stopping. Each action leads to a successor state, forming a game tree. Minimax assumes that the opponent will choose the action that is most unfavorable to the AI. At terminal states, a utility value represents the outcome, such as profit, market value obtained, budget remaining, or probability of winning. The maximizing player chooses the branch with the highest guaranteed utility, while the minimizing player chooses the branch with the lowest utility for the AI. Because the actual tree may be too large to search completely, the algorithm normally explores the tree only to a fixed depth and applies an evaluation function at the cutoff states.

ii. Computational challenges of large game/decision trees

The main difficulty is combinatorial growth. If the average branching factor is b and the search depth is d , a straightforward Minimax search may examine approximately b^d states, so even a moderate increase in the number of possible bids or rounds can make the search extremely expensive. In an auction, several bids may be legal at every stage and the market state changes after every action, further increasing the effective branching factor. Time and memory are also constrained because the agent must respond before the bidding deadline. A deeper search gives better strategic awareness but consumes more computation. A shallow search is faster but may suffer from the horizon effect, where an apparently good move hides an important consequence just beyond the search depth. Repeated or similar states can also cause redundant computation unless transposition tables or state caching are used. Therefore, the practical objective is not simply to search as deeply as possible, but to obtain the strongest decision within the available time.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning produces the same final decision as Minimax, but avoids exploring branches that cannot possibly change the result. Two bounds are maintained: **alpha**, the best value already guaranteed to the maximizing player, and **beta**, the best value already guaranteed to the minimizing player. At a maximizing node, alpha is updated when a better child value is found. At a minimizing node, beta is updated when a lower child value is found. Whenever alpha becomes greater than or equal to beta, the remaining children of that node can be discarded because the opponent already has a better alternative elsewhere in the tree.

Detailed reasoning: Suppose the AI is evaluating a MAX node and has already found a move worth 80, so $\alpha = 80$. It then enters an opponent-controlled MIN node. If one child of that MIN node gives the opponent a value of 60, β becomes 60. Since β (60) is already less than α (80), the MIN player can force an outcome no better than 60 from this branch. The MAX player already has an alternative worth 80, so the remaining children of the MIN node cannot make this branch preferable. They are pruned. With good move ordering, Alpha-Beta can reduce the effective work dramatically: in the best case its time complexity approaches $O(b^{d/2})$ rather than $O(b^d)$. This allows the agent to search substantially deeper within the same real-time limit.

iv. Evaluation function and justification

At a depth-limited cutoff, the AI needs a numerical evaluation of the current auction state. A suitable utility function could be:

Evaluation = 0.40(Expected Profit) + 0.20(Item/Market Value Advantage) + 0.15(Win Probability) + 0.10(Budget Safety) + 0.10(Future Opportunity) – 0.05(Risk/Overbidding Penalty).

The weights should be normalized so that each component is on a comparable scale. **Expected profit** is the primary factor because the system is intended to create financial value rather than merely win auctions. **Market value advantage** measures whether the current item is worth more to the AI than the price paid. **Win probability** captures the immediate competitive position. **Budget safety** prevents the agent from spending too much early and becoming unable to compete later. **Future opportunity** rewards preserving resources for valuable later rounds. Finally, a **risk/overbidding penalty** discourages bids whose expected benefit does not justify their financial exposure. In a real deployment, these weights should be learned or calibrated using historical simulations and validated against business objectives.

v. Recommended strategy for optimality vs real-time performance

A practical solution is **iterative-deepening Minimax with Alpha-Beta pruning**. The agent first searches to a shallow depth and obtains a valid action quickly. It then deepens the search while time remains, retaining the best completed result if the deadline is reached. Strong move ordering—examining high-probability or high-value bids first—makes Alpha-Beta pruning more effective. A transposition table should cache already evaluated states, while a compact heuristic should make leaf evaluation inexpensive. The system can also use a dynamic time budget: spend more computation in strategically critical rounds and less when the best action is obvious. This combination gives predictable real-time behavior while still exploiting additional computation whenever available. For safety, the final bidding policy should include hard constraints such as maximum bid, minimum expected return, and stop conditions independent of the search result.

2. Scenario: Autonomous Underwater Vehicle Route Planning

i. Greedy Best-First Search: behavior, strengths and weaknesses

Greedy Best-First Search selects the frontier node with the smallest heuristic estimate $h(n)$. In this AUV scenario, the heuristic can be the estimated straight-line distance from the current position to the last known survivor location. Therefore, the AUV tends to move toward locations that appear geographically closest to the survivor, without directly accounting for the movement cost already spent. Its major strength is speed: it often reaches a promising region quickly because it concentrates on the apparent goal direction. This is useful when rescue urgency is high. However, it is not generally optimal. A route that looks close in straight-line distance may pass through strong currents, deep water, debris, or poor-visibility regions and therefore have a much higher actual cost or risk. Because sonar information is partial and the environment changes, the heuristic can also become misleading. Greedy search may commit to an attractive direction and later require costly detours when an obstruction is discovered.

ii. How A* balances cost and heuristic

A* evaluates each frontier node using $f(n) = g(n) + h(n)$. Here, $g(n)$ is the actual cost accumulated from the AUV's starting position to node n , while $h(n)$ estimates the remaining cost from n to the survivor. The algorithm therefore considers both what has already been spent and what appears to remain. For example, a route that is slightly longer geometrically may be preferred if it avoids a powerful current and therefore has a much smaller total movement cost. If $h(n)$ is admissible—meaning it never overestimates the true remaining cost—and the cost model is appropriate, A* can produce an optimal path in a static environment. For the AUV, the cost function can be extended beyond distance to include energy consumption, current strength, depth-related effort, obstacle risk, and visibility. The result is a more meaningful route than one based on distance alone.

iii. Impact of heuristic accuracy

The heuristic strongly affects the behavior of both algorithms. Greedy Best-First Search depends almost entirely on $h(n)$, so an inaccurate heuristic can send the AUV toward a location that appears close but is actually difficult or dangerous to reach. A more informative heuristic generally reduces the amount of search, but Greedy still sacrifices guaranteed optimality. A* is more balanced because it combines $h(n)$ with $g(n)$. If $h(n)$ is admissible and consistent, A* retains its optimality properties while using the heuristic to prioritize promising nodes. If $h(n)$ is weak, A* remains correct but behaves more like uniform-cost search and may explore many nodes. If $h(n)$ overestimates, search may become faster but the usual optimality guarantee can be lost. In this application, straight-line distance is useful as a lower-bound estimate only when the cost model is compatible with that

assumption; otherwise it should be adjusted to account for currents, depth, and other known costs.

iv. Effect of shifting debris and changing visibility

Standard Greedy Best-First Search and standard A* are designed primarily for a fixed graph. The AUV scenario violates this assumption because sonar can reveal new obstacles, debris can move, and visibility can change. When a previously valid path becomes blocked, the search information associated with that path may become stale. Re-running the entire search from the current position is possible but can waste valuable time. A better implementation uses incremental or replanning methods so that only the affected part of the route is reconsidered. Algorithms such as D* Lite or Lifelong Planning A* are particularly suitable for environments in which edge costs and obstacle information change repeatedly. The navigation system should also continuously fuse sonar updates with the map and maintain safety margins around uncertain obstacles. Thus, dynamic replanning is not merely a performance optimization; it is essential for maintaining a valid and safe route.

v. Recommended algorithm and justification

For the described mission, **A* with dynamic replanning** is preferable to plain Greedy Best-First Search. The rescue objective requires fast decisions, but route quality, energy usage, and safety also matter. A* explicitly combines accumulated travel cost with an estimate of remaining cost, making it less likely to choose a geographically short but operationally expensive route. Because the environment changes, a practical system should use an incremental variant such as **D* Lite** when available, or repeatedly execute bounded A* searches when implementation simplicity is more important. The heuristic should be computationally cheap and conservative, and the cost function should include both travel effort and risk. A real-time safety layer should have authority to reject a search-generated route when sonar indicates a collision hazard or when uncertainty exceeds a safe threshold. This approach provides a strong balance of real-time responsiveness, near-optimal routing, adaptation to new information, and safety—making it more appropriate for an urgent autonomous rescue mission than an unmodified Greedy search.

Conclusion: The two scenarios illustrate the importance of selecting search methods according to the structure of the problem. For adversarial decision-making, Minimax with Alpha-Beta pruning and depth-limited evaluation provides strong strategic decisions under a time limit. For route planning, A* combines actual cost with heuristic guidance, while dynamic replanning is essential when the environment changes. In both cases, good heuristics, efficient pruning or replanning, and explicit safety/time constraints are critical for practical AI systems.